JobSheet - Week 5 – Generic Class

Nama: Zahra Azkiya Rahmani

NIM: 251524124

Kelas: 1D

Repo GitHub: https://github.com/yayyzea/1D_251524124_Teknik-Pemrograman-PR-.git

Instruksi Pengerjaan:

1. Kerjakan 6 soal di bawah ini dengan melengkapi setiap kolom jawaban yang disediakan pada jobsheet ini.
2. Jawaban setiap soal mencakup source code, screenshot hasil dari program yang ditampilkan full screen termasuk taskbar (tambahkan beberapa screenshot jika diperlukan), penjelasan permasalahan dan solusi yang dihadapi, nama teman yang membantu memecahkan masalah (opsional).
3. Dikumpulkan pada Assignment Classroom sesuai dengan deadline yang tertera pada assignment tersebut.
4. Format penamaan file jobsheet: W4_P_<Kelas 1X>_<3 Digit_NIM_Terakhir>.docx/pdf. Contoh: W5_P_1B_001.docx/pdf.
5. Submit semua jawaban dalam bentuk file java pada repository GitHub masing-masing.

No. 1 Try First

Soal Praktikum
Silahkan mencoba source code pada lampiran dan memberikan penjelasan detail setiap bagian pentingnya!

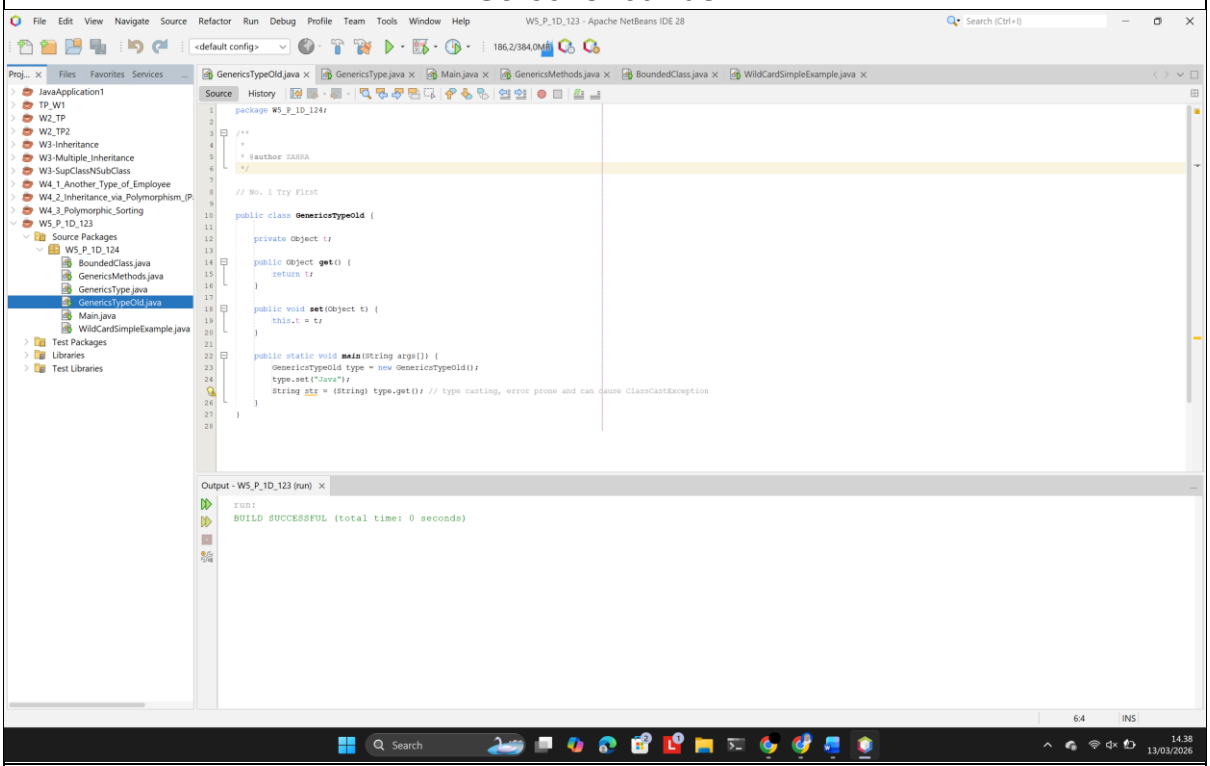
```
public class GenericsTypeOld {

    private Object t;

    public Object get() {
        return t;
    }

    public void set(Object t) {
        this.t = t;
    }

    public static void main(String args[]) {
        GenericsTypeOld type = new GenericsTypeOld();
```

```
        type.set("Java");
        String str = (String) type.get(); // type casting, error prone and can cause
ClassCastException
    }
}
```

Source Code

```
package W5_P_1D_124;

/**
 *
 * @author ZAHRA
 */

// No. 1 Try First

public class GenericsTypeOld {

    private Object t;

    public Object get() {
        return t;
    }

    public void set(Object t) {
        this.t = t;
    }

    public static void main(String args[]) {
        GenericsTypeOld type = new GenericsTypeOld();
        type.set("Java");
        String str = (String) type.get(); // type casting, error prone and can cause ClassCastException
    }
}
```

Screenshot Hasil



Penjelasan

- private Object t: Field bertipe Object, sehingga dapat menyimpan nilai bertipe apapun.
- public Object get(): Method ini mengembalikan nilai dengan tipe Object dan pemanggil harus melakukan type casting secara manual.
- public void set(Object t): Menerima parameter bertipe Object sehingga dapat menerima nilai apapun.

- String str = (String) type.get():Type casting dari Object ke String.

Program ini berhasil dijalankan (tidak ada *error* kompilasi) namun tidak menghasilkan *output* apa-apa di layar karena tidak ada perintah "System.out.println". Kode ini menjelaskan cara menyimpan berbagai tipe data menggunakan kelas Object. Karena pengembaliannya berupa Object, diperlukan *type casting* secara manual, seperti "(String) type.get()". Pendekatan ini sangat rawan *error* (*error prone*) dan bisa menyebabkan ClassCastException saat program dijalankan (*runtime*) jika tipe data yang dimasukkan tidak sesuai dengan yang di-*casting.*

Nama Teman Hal yang Dibantu (Opsional)

No. 2 Using Generic Class

Soal Praktikum

Silahkan mencoba source code pada lampiran dan memberikan penjelasan detail setiap bagian pentingnya!

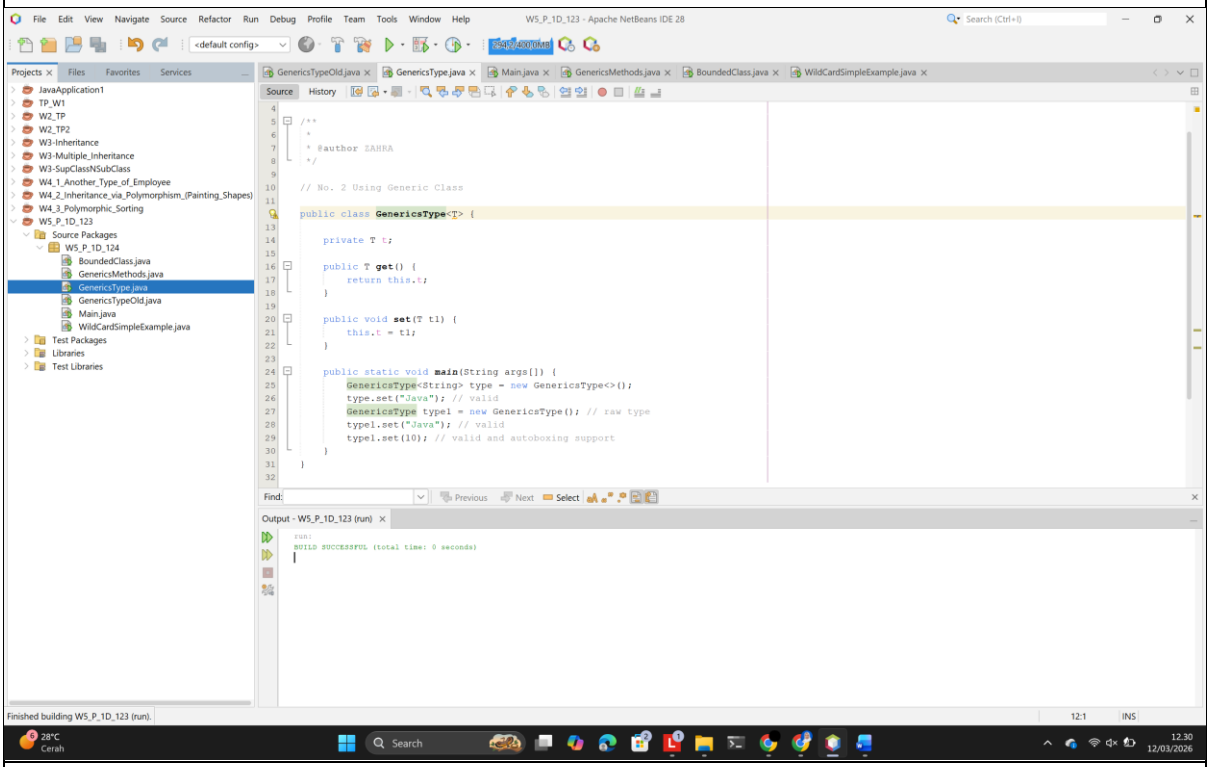
```
public class GenericsType<T> {

    private T t;

    public T get() {
        return this.t;
    }

    public void set(T t1) {
        this.t = t1;
    }

    public static void main(String args[]) {
        GenericsType<String> type = new GenericsType<>();
        type.set("Java"); // valid

        GenericsType type1 = new GenericsType(); // raw type
        type1.set("Java"); // valid
        type1.set(10); // valid and autoboxing support
    }
}
```

Source Code

```
package W5_P_1D_124;



/**
 *
 * @author ZAHRA
 */

// No. 2 Using Generic Class

public class GenericsType<T> {

    private T t;

    public T get() {
        return this.t;
    }

    public void set(T t1) {
        this.t = t1;
    }

    public static void main(String args[]) {
        GenericsType<String> type = new GenericsType<>();
        type.set("Java"); // valid
        GenericsType type1 = new GenericsType(); // raw type
        type1.set("Java"); // valid
        type1.set(10); // valid and autoboxing support
    }
```

}

Screenshot Hasil



Penjelasan

- public class GenericsType<T>: Deklarasi Generic Class dengan parameter tipe T.
- private T t: Field menggunakan tipe generik T, bukan Object seperti sebelumnya.
- GenericsType<String> type = new GenericsType<>(): Membuat objek dengan tipe String.
- GenericsType type1 = new GenericsType(): Penggunaan raw type (tanpa parameter tipe)
- type1.set(10): Dengan raw type, Integer dapat dimasukkan dan Java melakukan autoboxing dari int 10 menjadi Integer.

Program ini tidak mencetak *output* karena tidak ada perintah *print*. Kode ini menunjukkan penggunaan Generic Class dengan parameter <T>. Dengan menggunakan *Generic*, tipe data dapat ditentukan secara spesifik apa saja yang boleh dimasukkan sejak awal, sehingga jauh lebih aman dari *error*. Di sisi lain, instansiasi tanpa tanda kurung sudut (GenericsType type1) disebut sebagai *raw type*. *Raw type* dapat menerima tipe data campuran (String, lalu ditimpa Integer 10 berkat dukungan *autoboxing*), persis seperti cara kerja kelas Object versi lama.

Nama Teman Hal yang Dibantu (Opsional)

No. 3 Java Generic Class

Soal Praktikum

Silahkan mencoba source code pada lampiran dan memberikan penjelasan detail setiap bagian pentingnya!

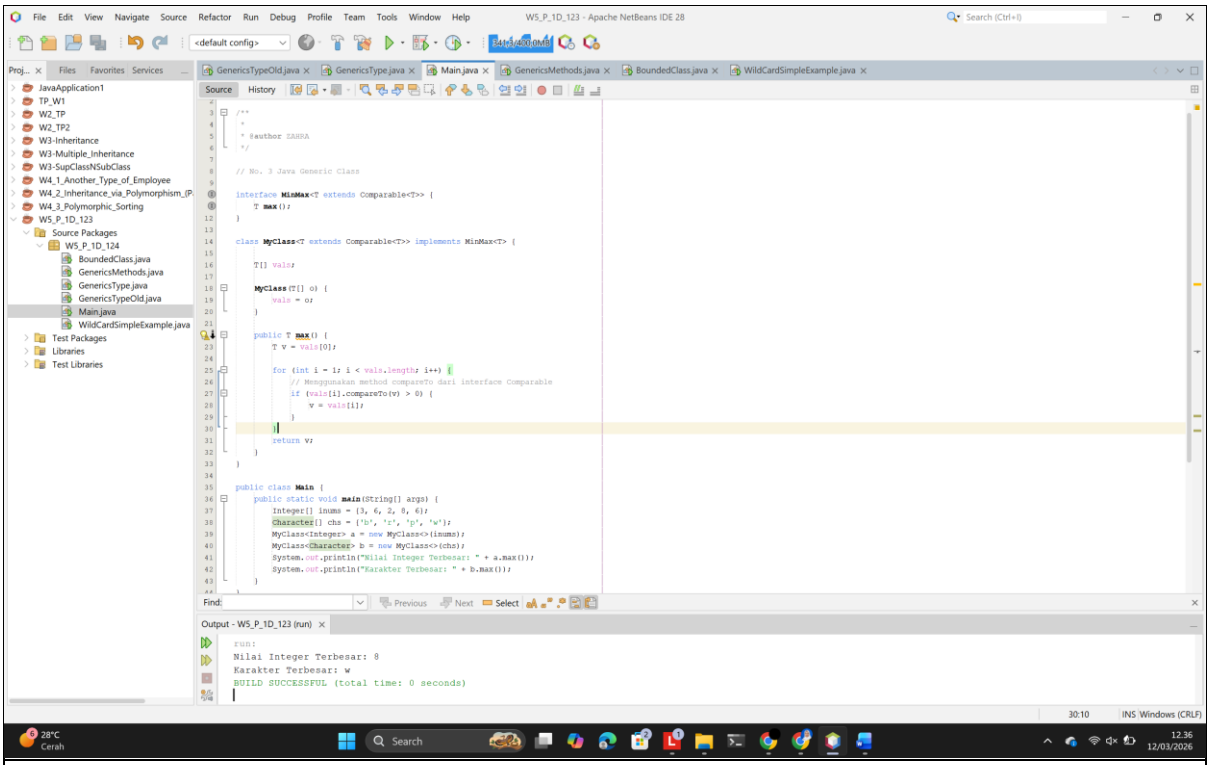
```
interface MinMax<T extends Comparable<T>> {
    T max();
}

class MyClass<T extends Comparable<T>> implements MinMax<T> {
    T[] vals;

    MyClass(T[] o) {
        vals = o;
    }

    public T max() {
        T v = vals[0];

        for (int i = 1; i < vals.length; i++) {
            // Menggunakan method compareTo dari interface Comparable
            if (vals[i].compareTo(v) > 0) {
                v = vals[i];
            }
        }
        return v;
    }
}

public class Main {
    public static void main(String[] args) {
        Integer[] inums = {3, 6, 2, 8, 6};
        Character[] chs = {'b', 'r', 'p', 'w'};

        MyClass<Integer> a = new MyClass<>(inums);
        MyClass<Character> b = new MyClass<>(chs);

        System.out.println("Nilai Integer Terbesar: " + a.max());
        System.out.println("Karakter Terbesar: " + b.max());
    }
}
```

Source Code

```
package W5_P_1D_124;

/**
 *
 * @author ZAHRA
 */
```

```
// No. 3 Java Generic Class

interface MinMax<T extends Comparable<T>> {
    T max();
}

class MyClass<T extends Comparable<T>> implements MinMax<T> {

    T[] vals;

    MyClass(T[] o) {
        vals = o;
    }

    public T max() {
        T v = vals[0];

        for (int i = 1; i < vals.length; i++) {
            // Menggunakan method compareTo dari interface Comparable
            if (vals[i].compareTo(v) > 0) {
                v = vals[i];
            }
        }
        return v;
    }
}

public class Main {
    public static void main(String[] args) {
        Integer[] inums = {3, 6, 2, 8, 6};
        Character[] chs = {'b', 'r', 'p', 'w'};
        MyClass<Integer> a = new MyClass<>(inums);
        MyClass<Character> b = new MyClass<>(chs);
        System.out.println("Nilai Integer Terbesar: " + a.max());
        System.out.println("Karakter Terbesar: " + b.max());
    }
}
```

Screenshot Hasil



Penjelasan
• interface MinMax<T extends Comparable<T>>: Mendefinisikan interface Generic dengan bounded type parameter danT harus mengimplementasi Comparable<T>. • T extends Comparable<T>: Upper bounded type parameter, sehingga hanya tipe yang mengimplementasi Comparable (seperti Integer, String, Character) yang dapat digunakan. • public T max(): Method dalam interface yang mengembalikan nilai maksimum bertipe T. • MyClass<T extends Comparable<T>> implements MinMax<T>: Kelas konkret yang mengimplementasi interface MinMax dengan tipe generic yang sama. • vals[i].compareTo(v) > 0: Menggunakan method compareTo dari Comparable. Jika hasilnya > 0 berarti vals[i] lebih besar dari v, maka v diganti. Program ini mendemonstrasikan Bounded Type Parameters (upper bound) pada Generic Interface dan Class. Sintaks “<T extends Comparable<T>>” memastikan bahwa tipe data T yang diteruskan ke kelas ini harus mengimplementasikan interface Comparable. Bound diperlukan agar method compareTo() dapt digunakan dalam method max() untuk membandingkan elemen-elemen array dan mencari nilai terbesarnya. Kelas MyClass ini terbukti fleksibel karena dapat menangani array Integer maupun Character.
Nama Teman Hal yang Dibantu (Opsional)

No. 4 Java Generic Method

Soal Praktikum

Silahkan mencoba source code pada lampiran dan memberikan penjelasan detail setiap bagian pentingnya!

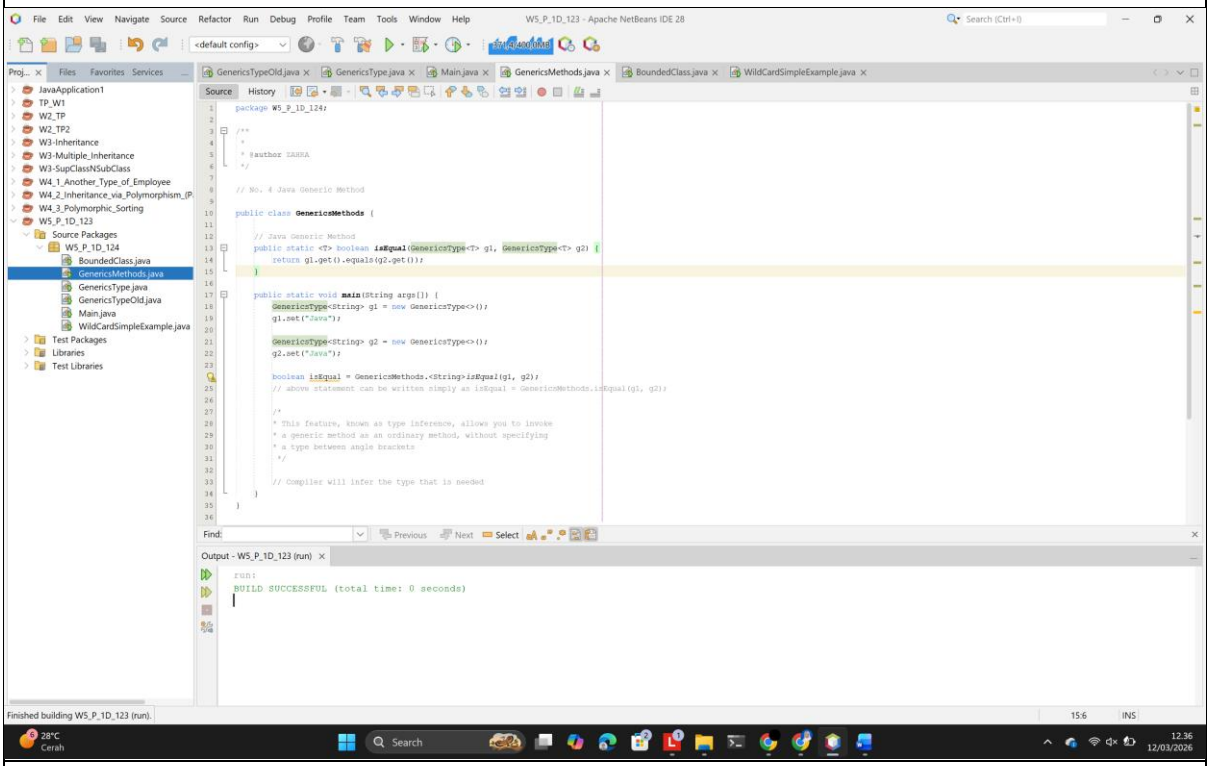
```
public class GenericsMethods {

   // Java Generic Method
   public static <T> boolean isEqual(GenericsType<T> g1, GenericsType<T> g2) {
      return g1.get().equals(g2.get());
   }

   public static void main(String args[]) {
      GenericsType<String> g1 = new GenericsType<>();
      g1.set("Java");

      GenericsType<String> g2 = new GenericsType<>();
      g2.set("Java");

      boolean isEqual = GenericsMethods.<String>isEqual(g1, g2);
      // above statement can be written simply as
      isEqual = GenericsMethods.isEqual(g1, g2);

      /*
       * This feature, known as type inference, allows you to invoke
       * a generic method as an ordinary method, without specifying
       * a type between angle brackets
       */

      // Compiler will infer the type that is needed
   }
}
```

Source Code

```
package W5_P_1D_124;

/**
 *
 * @author ZAHRA
 */

// No. 4 Java Generic Method

public class GenericsMethods {

   // Java Generic Method
   public static <T> boolean isEqual(GenericsType<T> g1, GenericsType<T> g2) {
      return g1.get().equals(g2.get());
   }

   public static void main(String args[]) {
      GenericsType<String> g1 = new GenericsType<>();
      g1.set("Java");

      GenericsType<String> g2 = new GenericsType<>();
      g2.set("Java");

      boolean isEqual = GenericsMethods.<String>isEqual(g1, g2);
      // above statement can be written simply as isEqual = GenericsMethods.isEqual(g1, g2);

      /*
```

```
            * This feature, known as type inference, allows you to invoke
            * a generic method as an ordinary method, without specifying
            * a type between angle brackets
             */

            // Compiler will infer the type that is needed
        }
}
```

Screenshot Hasil



Penjelasan

- public static <T> boolean isEqual(...): Generic Method di mana parameter tipe <T> dideklarasikan sebelum return type boolean, sehingga method ini bisa digunakan untuk tipe apapun.
- GenericsType<T> g1, GenericsType<T> g2: Kedua parameter harus memiliki tipe generi yang sama T, memastikan perbandingan hanya dilakukan antara objek sejenis.
- g1.get().equals(g2.get()): Membandingkan isi kedua objek GenericsType menggunakan method equals().
- GenericsMethods.<String>isEqual(g1, g2): Explicit type argument.
- GenericsMethods.isEqual(g1, g2): Type Inference .

Program ini menunjukkan Generic Method di Java, Method isEqual dideklarasikan dengan parameter tipe <T> di depan tipe kembaliannya boolean. Method ini membandingkan isi dari dua objek GenericsType dan fitur Type Inference dapat mengetahui tipe data yang dibutuhkan. Oleh karena itu, pemanggilan method tidak harus selalu ditulis lengkap “<String>isEqual(g1, g2)” melainkan dapat ditulis seperti method biasa “isEqual(g1, g2)” tanpa tanda kurung sudut <>.

Nama Teman Hal yang Dibantu (Opsional)

No. 5 Java Generics Bounded Type Parameters

Soal Praktikum
Silahkan mencoba source code pada lampiran dan memberikan penjelasan detail setiap bagian pentingnya!

```
class Bound<T extends A> {

    private T objRef;

    public Bound(T obj) {
        this.objRef = obj;
    }

    public void doRunTest() {
        this.objRef.displayClass();
    }
}

class A {

    public void displayClass() {
        System.out.println("Inside super class A");
    }
}

class B extends A {

    public void displayClass() {
        System.out.println("Inside sub class B");
    }
}

class C extends A {

    public void displayClass() {
        System.out.println("Inside sub class C");
    }
}

public class BoundedClass {

    public static void main(String a[]) {

        // Creating object of sub class C and
        // passing it to Bound as a type parameter.
        Bound<C> bec = new Bound<C>(new C());
        bec.doRunTest();

        // Creating object of sub class B and
```

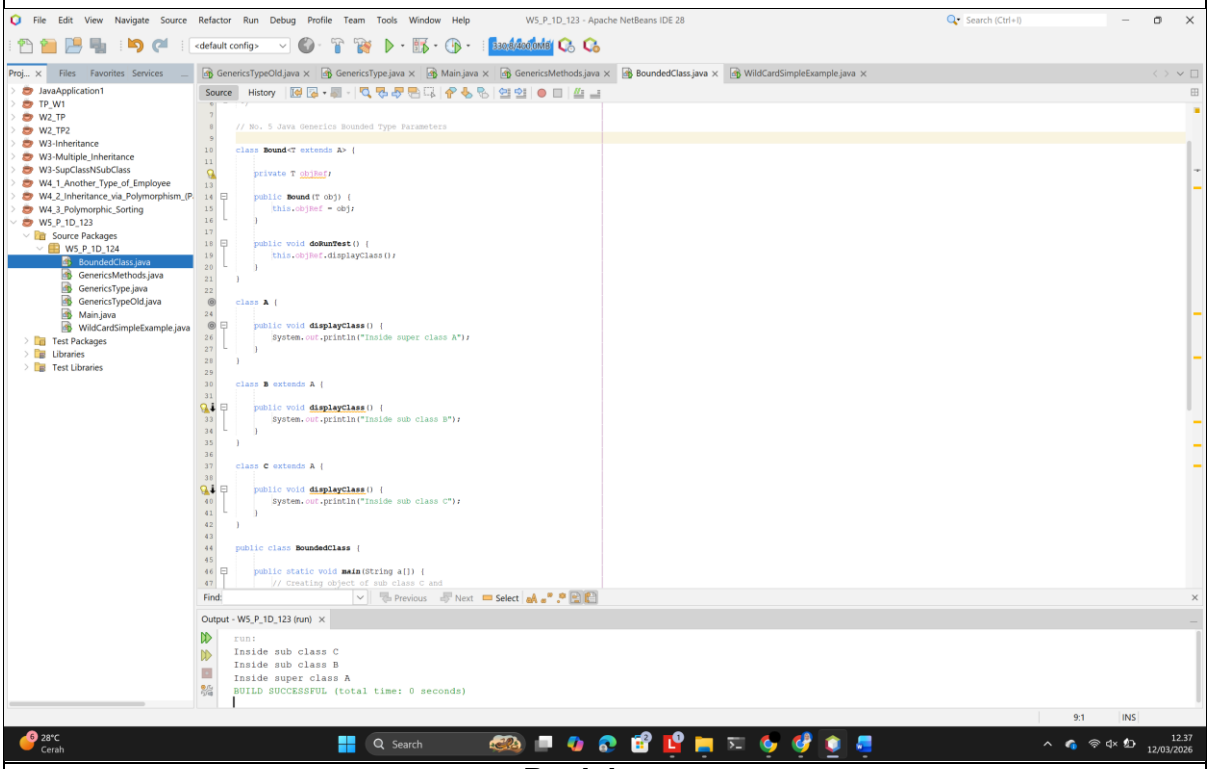
```
        // passing it to Bound as a type parameter.
        Bound<B> beb = new Bound<B>(new B());
        beb.doRunTest();

        // similarly passing super class A
        Bound<A> bea = new Bound<A>(new A());
        bea.doRunTest();
    }
}
```

Source Code

```
package W5_P_1D_124;

/**
 *
 * @author ZAHRA
 */

// No. 5 Java Generics Bounded Type Parameters

class Bound<T extends A> {

    private T objRef;

    public Bound(T obj) {
        this.objRef = obj;
    }

    public void doRunTest() {
        this.objRef.displayClass();
    }
}

class A {

    public void displayClass() {
        System.out.println("Inside super class A");
    }
}

class B extends A {

    public void displayClass() {
        System.out.println("Inside sub class B");
    }
}

class C extends A {

    public void displayClass() {
        System.out.println("Inside sub class C");
    }
}

public class BoundedClass {

    public static void main(String a[]) {
        // Creating object of sub class C and
        // passing it to Bound as a type parameter.
        Bound<C> bec = new Bound<C>(new C());
        bec.doRunTest();
        // Creating object of sub class B and
        // passing it to Bound as a type parameter.
```

```
        Bound<B> beb = new Bound<B>(new B());
        beb.doRunTest();
        // similarly passing super class A
        Bound<A> bea = new Bound<A>(new A());
        bea.doRunTest();
    }
}
```

Screenshot Hasil



Penjelasan

- class Bound<T extends A>: Bounded Type Parameter dengan upper bound class A. Tipe T hanya boleh berupa class A itu sendiri atau subclass-nya (B atau C).
- private T objRef: Field generic yang menyimpan referensi ke objek bertipe T.
- this.objRef.displayClass():Method memanggil fungsi displayClass() dari objek.

Program ini merupakan Bounded Type Parameters menggunakan kelas. Sintaks “class Bound<T extends A>” membatasi bahwa kelas Bound hanya mau menerima tipe data A atau *sub-class* turunan dari A (yaitu kelas B dan C), sehingga objek yang masuk ke dalam Bound pasti memiliki method displayClass() dari kelas A. Jadi, saat method doRunTest() dieksekusi, sifat polimorfisme akan berjalan sesuai dengan objek (C, B, atau A) yang dilewatkan.

Nama Teman Hal yang Dibantu (Opsional)

No. 6 Java Generic WildCard

Soal Praktikum

Silahkan mencoba source code pada lampiran dan memberikan penjelasan detail setiap bagian pentingnya!

```
import java.util.ArrayList;
import java.util.Collection;
import java.util.HashSet;
import java.util.LinkedList;

/**
 * Wildcard Arguments With An Unknown Type
 * @author javaguides.net
 **/
public class WildCardSimpleExample {

    public static void printCollection(Collection<?> c) {
        for (Object e : c) {
            System.out.println(e);
        }
    }

    public static void main(String[] args) {
        Collection<String> collection = new ArrayList<>();
        collection.add("ArrayList Collection");
        printCollection(collection);

        Collection<String> collection2 = new LinkedList<>();
```

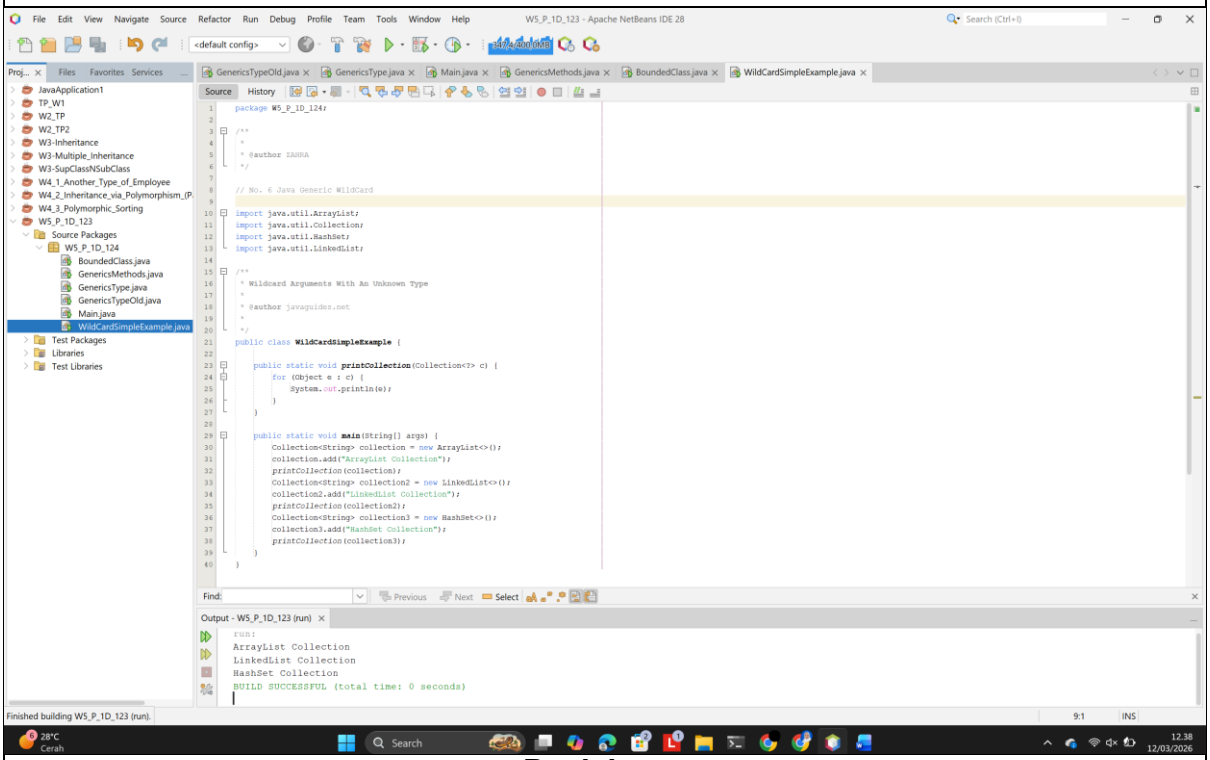
```
        collection2.add("LinkedList Collection");
        printCollection(collection2);

        Collection<String> collection3 = new HashSet<>();
        collection3.add("HashSet Collection");
        printCollection(collection3);
    }
}
```

Source Code

```
package W5_P_1D_124;

/**
 *
 * @author ZAHRA
 */

// No. 6 Java Generic WildCard

import java.util.ArrayList;
import java.util.Collection;
import java.util.HashSet;
import java.util.LinkedList;

/**
 * Wildcard Arguments With An Unknown Type
 *
 * @author javaguides.net
 *
 */
public class WildCardSimpleExample {
```

```
        public static void printCollection(Collection<?> c) {
            for (Object e : c) {
                System.out.println(e);
            }
        }

        public static void main(String[] args) {
            Collection<String> collection = new ArrayList<>();
            collection.add("ArrayList Collection");
            printCollection(collection);
            Collection<String> collection2 = new LinkedList<>();
            collection2.add("LinkedList Collection");
            printCollection(collection2);
            Collection<String> collection3 = new HashSet<>();
            collection3.add("HashSet Collection");
            printCollection(collection3);
        }
}
```

Screenshot Hasil



Penjelasan

- Collection<?>: Method dapat menerima Collection dari tipe apa pun.
- for (Object e : c): Karena tipe tidak diketahui, maka elemen dibaca sebagai Object.
- printCollection(c1): Memanggil method yang sama untuk ArrayList, LinkedList, dan HashSet (semua diterima karena parameter menggunakan wildcard).

Program ini menerapkan penggunaan Wildcard sederhana dengan simbol tanda tanya <?>. Method printCollection(Collection<?> c) dapat menerima objek Collection dengan *tipe elemen apa saja*. Perulangan for-each di dalamnya menggunakan tipe Object e untuk mencetak nilai-nilai di dalam koleksi tersebut. Pada method *main*, terbukti bahwa method wildcard ini berhasil menerima berbagai implementasi Collection (ArrayList, LinkedList, maupun HashSet).

Nama Teman Hal yang Dibantu (Opsional)